

Dynamic Programming

This notebook illustrates the basic algorithms of dynamic programming using the classical “gridworld” environment. We first consider the deterministic case where the environment is known, i.e. the actions that are selected are actually carried out, and the rewards are known. Next, we consider the case where there is some uncertainty regarding the outcome of an action: the agent chooses an action, but the actual action that is carried out is different from the one that was chosen. We however know the probabilities of the different outcomes. In another notebook, we will consider the case where the agent does not know the transition probabilities and has to learn them from experience.

The test case

The environment is a 6x6 grid, where the agent starts at the top-left corner (state S_0) and aims to reach the bottom-right corner (state S_{35}). The agent can move in four directions: up, down, left, and right. However, there are obstacles (walls) at states S_{14} , S_{20} , S_{21} and S_{27} that the agent cannot pass through, and a pit at state S_{16} where the agent would get stuck. The objective is modeled by assigning a reward to each state: the pit has a reward of -1000, and the end goal a reward of 10. Passing through any other state has a reward of -1. The agent receives the reward immediately upon entering a state. The episode ends when the agent reaches either the end goal or the pit. Maximizing the reward is equivalent to finding the shortest feasible path to the goal.

Gridworld with obstacles and a pit

S0 (Start)	S1	S2	S3	S4	S5
S6	S7	S8	S9	S10	S11
S12	S13	S14 (Wall)	S15	S16 (Pit)	S17
S18	S19	S20 (Wall)	S21 (Wall)	S22	S23
S24	S25	S26	S27 (Wall)	S28	S29
S30	S31	S32	S33	S34	S35 (Goal)

Solution of the Bellman value equation by backward recursion

The value of state s is the maximum over all possible action of the sum of the current reward plus the discounted value of the resulting next state:

$$V(s) = \max_a (R(s, a) + \gamma V(s'))$$

where:

- $R(s, a)$ reward received when taking action a in state s ,
- γ discount factor, and
- s' next state resulting from taking action a in state s .

We repeatedly use the following helper function to compute the next state s' and reward $R(s, a)$ for a given state and action.

```
def move(s, a_idx):
    """
    Returns the next state and reward for taking action a_idx in state s.
    """
    row, col = divmod(s, GRID_SIZE)
```

```

dr, dc = ACTIONS[a_idx]
nr = max(0, min(GRID_SIZE - 1, row + dr))
nc = max(0, min(GRID_SIZE - 1, col + dc))
next_s = nr * GRID_SIZE + nc

if next_s in OBSTACLES:
    next_s = s

if next_s in GOAL_STATES:
    reward = GOAL_REWARD
elif next_s in PIT_STATES:
    reward = PIT_REWARD
else:
    reward = -1

return next_s, reward

```

It is convenient to treat the Gridworld as a directed graph, where an edge exists from state s to state s' if there is an action that can take the agent from s to s' . The reward for taking that action is the reward of the resulting state s' .

```

# Grid Setup
TERMINAL_STATES = {35: GOAL_REWARD, 16: PIT_REWARD} # {state: reward}
GAMMA = 0.9
ACTIONS = [(-1, 0), (1, 0), (0, -1), (0, 1)] # Up, Down, Left, Right

# Precompute Predecessors (The Graph)
predecessors = {i: [] for i in range(STATES)}
for s in range(STATES):
    if s in TERMINAL_STATES or s in OBSTACLES:
        continue
    r, c = divmod(s, GRID_SIZE)
    for a in ACTIONS:
        nr = max(0, min(GRID_SIZE - 1, r + a[0]))
        nc = max(0, min(GRID_SIZE - 1, c + a[1]))
        ns = nr * GRID_SIZE + nc
        if ns != s and ns not in OBSTACLES:
            predecessors[ns].append(s)

```

As an illustration, the predecessors of the states in the first row are:

S0: [1, 6]

```

S1: [0, 2, 7]
S2: [1, 3, 8]
S3: [2, 4, 9]
S4: [3, 5, 10]
S5: [4, 11]

```

We can now use backward recursion to solve the Bellman value equation. We start from the terminal states and propagate their values backward to their predecessors, updating the value of each state according to the Bellman update rule. This process continues until we have updated all reachable states.

There are cycles in the graph, but a path that includes a cycle will never be optimal, since each step incurs a cost of -1. Since we only update a state if we find a better value, the recursion will eventually terminate when no more updates are possible. The value of the initial state S_0 will be the optimal value for that state, and the values of all other states will reflect the optimal return from those states under the optimal policy. The result is stored in the value table V. With this value table, we can derive the optimal path by choosing the action that leads to the neighboring state with the highest value.

```

# Value Table
V = np.full(STATES, -np.inf)
for s, r in TERMINAL_STATES.items():
    V[s] = r

def backward_update(current_state):
    """
    Recursively propagates value from a state to its ancestors.
    """
    current_val = V[current_state]
    for prev in predecessors[current_state]:
        # Bellman: V(prev) = max(V(prev), step_reward + gamma * V(current))
        step_reward = -1
        new_val = step_reward + GAMMA * current_val

        if new_val > V[prev]:
            V[prev] = new_val
            # Continue recursion backward only if we found a better path
            backward_update(prev)

# backward recursion, starting at the terminal state
backward_update(GOAL_STATES[0])

```

Optimal State Values:

$$V = \begin{bmatrix} -3.0 & -2.3 & -1.4 & -0.4 & 0.6 & 1.8 \\ -2.3 & -1.4 & -0.4 & 0.6 & 1.8 & 3.1 \\ -1.4 & -0.4 & \text{Wall} & -0.4 & -1000.0 & 4.6 \\ -0.4 & 0.6 & \text{Wall} & \text{Wall} & 4.6 & 6.2 \\ 0.6 & 1.8 & 3.1 & \text{Wall} & 6.2 & 8.0 \\ 1.8 & 3.1 & 4.6 & 6.2 & 8.0 & 10.0 \end{bmatrix}$$

We can now determine the optimal path from the initial state to the goal, using the predecessor table computed earlier.

```
def get_next_states(s):
    """Returns valid next states from state s."""
    next_states = []
    for a in ACTIONS:
        r, c = divmod(s, GRID_SIZE)
        nr = max(0, min(GRID_SIZE - 1, r + a[0]))
        nc = max(0, min(GRID_SIZE - 1, c + a[1]))
        ns = nr * GRID_SIZE + nc
        if ns != s and ns not in OBSTACLES and ns <= STATES - 1:
            next_states.append(ns)
    return next_states

# Extracting the optimal path from S0 to the goal
current_state = 0
optimal_path = [current_state]
while current_state != GOAL_STATES[0]:
    next_states = get_next_states(current_state)
    if not next_states:
        break
    max_index, max_value = max(
        enumerate(V[next_states]),
        key=lambda x: x[1],
    )
    current_state = next_states[max_index]
    optimal_path.append(current_state)
```

One optimal path is pictured below.

Optimal Path (Deterministic)

S0 (Start)	S1	S2	S3	S4	S5
S6	S7	S8	S9	S10	S11
S12	S13	S14 (Wall)	S15	S16 (Pit)	S17
S18	S19	S20 (Wall)	S21 (Wall)	S22	S23
S24	S25	S26	S27 (Wall)	S28	S29
S30	S31	S32	S33	S34	S35 (Goal)

Solution of the Bellman Optimality equation

In the deterministic case, the MDP action-value function $Q(s, a)$ is defined as the return of taking action a in state s and thereafter following the optimal policy:

$$Q(s, a) = R(s, a) + \gamma \max_{a'} Q(s', a')$$

where s' is the next state resulting from taking action a in state s . The state value function $V(s)$ is the optimal over all actions:

$$V(s) = \max_a Q(s, a)$$

We look for the optimal value function $Q^*(s, a)$, which satisfies:

$$Q^*(s, a) = R(s, a) + \gamma \max_{a'} Q^*(s', a')$$

The optimal policy $\pi^*(s)$ will then be given by:

$$\pi^*(s) = \arg \max_a Q^*(s, a)$$

Let $\mathcal{Q}$ be the space of all possible Q-tables (bounded real-valued functions over $S \times A$). We define the Bellman operator $\mathcal{T}$ as:

$$(\mathcal{T}Q)(s, a) = R(s, a) + \gamma \max_{a'} Q(s', a')$$

This operator takes an existing Q-table and produces a new one by performing a one-step lookahead.

We next show that $\mathcal{T}$ is a contraction mapping.

Consider two Q-tables Q_1 and Q_2 . For any (s, a) :

$$|(\mathcal{T}Q_1)(s, a) - (\mathcal{T}Q_2)(s, a)| = \gamma |\max_{a'} Q_1(s', a') - \max_{a'} Q_2(s', a')|$$

Using the fact that $|\max_x f(x) - \max_x g(x)| \leq \max_x |f(x) - g(x)|$, we can further bound this by:

$$|(\mathcal{T}Q_1)(s, a) - (\mathcal{T}Q_2)(s, a)| \leq \gamma \max_{a'} |Q_1(s', a') - Q_2(s', a')|$$

or,

$$\|\mathcal{T}Q_1 - \mathcal{T}Q_2\|_{\infty} \leq \gamma \|Q_1 - Q_2\|_{\infty}$$

Since $\gamma < 1$, this shows that $\mathcal{T}$ is a contraction mapping with respect to the supremum norm. By the Banach fixed-point theorem, this implies that $\mathcal{T}$ has a unique fixed point in $\mathcal{Q}$, and that iteratively applying $\mathcal{T}$ will converge to this fixed point.

We start with the initial guess $Q(s, a) = 0$ and repeatedly applying the update rule until convergence:

$$Q_{k+1}(s, a) \leftarrow R(s, a) + \gamma \max_{a'} Q_k(s', a')$$

As expected, the optimal Q-values are the same as the optimal state values computed earlier.

```
THRESHOLD = 1e-10
# Initialize Q-table
Q = np.zeros((STATES, len(ACTIONS)))

# 1. Iteration Loop
iter_count = 0
while True:
    delta = 0
    Q_old = Q.copy()

    for s in range(STATES):
        if s in TERMINAL_STATES or s in OBSTACLES:
```

```

        continue

    for a_idx in range(len(ACTIONS)):
        next_s, reward = move(s, a_idx)

        # The Q-update: reward + discounted max value of next state
        new_q = reward + GAMMA * np.max(Q_old[next_s])

        delta = max(delta, abs(new_q - Q[s, a_idx]))
        Q[s, a_idx] = new_q

    if delta < THRESHOLD:
        break

    iter_count += 1

print(f"Converged after {iter_count} iterations.")

# 2. Policy Extraction
# argmax across the action dimension (axis 1)
policy = np.argmax(Q, axis=1)
print("Optimal Policy (0:U, 1:D, 2:L, 3:R):")
print(policy.reshape(GRID_SIZE, GRID_SIZE))

# 3. Derive the optimal path from S0 to goal using the optimal policy
current_state = 0
optimal_path = [current_state]
visited_states = set([current_state])
while current_state not in GOAL_STATES:
    action_idx = policy[current_state]
    next_state, _ = move(current_state, action_idx)
    optimal_path.append(next_state)
    if current_state == next_state or next_state in visited_states:
        break # Stop if it bumps into walls intentionally or cycles
    visited_states.add(next_state)
    current_state = next_state
print("Intended optimal path from S0:", optimal_path)

```

```

Converged after 11 iterations.
Optimal Policy (0:U, 1:D, 2:L, 3:R):
[[1 1 1 1 1 1]
 [1 1 3 3 3 1]

```



```
[1 1 0 0 0 1]
[1 1 0 0 1 1]
[1 1 1 0 1 1]
[3 3 3 3 3 0]]
```

Intended optimal path from S0: [0, 6, 12, 18, 24, 30, 31, 32, 33, 34, 35]

Stochastic Bellman Optimality Equation

Let's now consider the situation where the moves are not fully predictable. The agent intends to move in a certain direction, but the actual move is subject to some randomness. The transition probabilities are given by the following table:

Intended Action	Probability of move			
	up	down	left	right
Up	80%	0%	10%	10%
Down	0%	80%	10%	10%
Left	10%	10%	80%	0%
Right	10%	10%	0%	80%

The Bellman optimality equation for the action-value function $Q(s, a)$ now involves the expected reward over all possible next states and is given by:

$$Q(s, a) = \sum_{s'} p(s'|s, a) \left[R(s, a, s') + \gamma \max_{a'} Q(s', a') \right]$$

where $p(s'|s, a)$ is the probability of transitioning to state s' when taking action a in state s , and $R(s, a, s')$ is the reward received for that transition. The optimal policy $\pi^*(s)$ can still be derived from the optimal Q-values as:

$$\pi^*(s) = \arg \max_a Q(s, a)$$

The iterative update rule for Q-values in the stochastic case becomes:

$$Q_{k+1}(s, a) \leftarrow \sum_{s'} p(s'|s, a) \left[R(s, a, s') + \gamma \max_{a'} Q_k(s', a') \right]$$

```
# Initialize Q-table
Q = np.zeros((STATES, len(ACTIONS)))

def get_transition_results(s, a_idx):
    """
```

```

Returns a list of (probability, next_state, reward)
"""

# No transition from terminal states
if s in GOAL_STATES or s in OBSTACLES:
    return [(1.0, s, 0)]

results = []
# Map intended action to side-effect actions
slippage = {
    0: [2, 3], # Up -> Left, Right
    1: [2, 3], # Down -> Left, Right
    2: [0, 1], # Left -> Up, Down
    3: [0, 1], # Right -> Up, Down
}

# 80% Intended Action
results.append((0.8, *move(s, a_idx)))
# 10% Slip A
results.append((0.1, *move(s, slippage[a_idx][0])))
# 10% Slip B
results.append((0.1, *move(s, slippage[a_idx][1])))

return results

# 2. Stochastic Q-Iteration Loop
iter_count = 0
while True:
    delta = 0
    Q_old = Q.copy()

    for s in range(STATES):
        if s in GOAL_STATES or s in OBSTACLES:
            continue

        for a_idx in range(len(ACTIONS)):
            # Calculate Expected Value: Sum of p * (r + gamma * maxQ)
            expected_value = 0
            outcomes = get_transition_results(s, a_idx)

            for prob, next_s, reward in outcomes:

```

```

        expected_value += prob * (
            reward + GAMMA * np.max(Q_old[next_s])
        )

    delta = max(delta, abs(expected_value - Q[s, a_idx]))
    Q[s, a_idx] = expected_value

    if delta < THRESHOLD:
        break
    iter_count += 1

print(f"Converged after {iter_count} iterations.")

# 3. Policy Extraction
policy = np.argmax(Q, axis=1).reshape(GRID_SIZE, GRID_SIZE)
print("Optimal Policy in a Stochastic World:")
print(policy)

```

```

Converged after 110 iterations.
Optimal Policy in a Stochastic World:
[[1 1 2 2 3 1]
 [1 1 2 2 0 1]
 [1 1 0 2 1 3]
 [1 1 0 0 1 1]
 [1 1 1 0 1 1]
 [3 3 3 3 3 0]]

```

The figure below shows the optimal policy in the stochastic case, where the arrows indicate the optimal action for each state, and the value of the best action is displayed below the arrow.

Stochastic Gridworld: Risk-Averse Policy

S0 ↓ V:-4.0	S1 ↓ V:-3.7	S2 ← V:-4.4	S3 ← V:-5.1	S4 → V:-5.2	S5 ↓ V:-4.5
S6 ↓ V:-3.2	S7 ↓ V:-2.7	S8 ← V:-3.7	S9 ← V:-4.8	S10 ↑ V:-5.5	S11 ↓ V:-3.7
S12 ↓ V:-2.3	S13 ↓ V:-1.6	WALL	S15 ← V:-7.5	PIT	S17 → V:-2.6
S18 ↓ V:-1.3	S19 ↓ V:-0.3	WALL	WALL	S22 ↓ V:-1.2	S23 ↓ V:-0.8
S24 ↓ V:-0.2	S25 ↓ V:-1.2	S26 ↓ V:-2.7	WALL	S28 ↓ V:-1.2	S29 ↓ V:-0.3
S30 → V:1.0	S31 → V:2.7	S32 → V:4.6	S33 → V:6.9	S34 → V:9.3	GOAL

Of particular interest are the optimal decisions in S_{15} and S_{17} , which are adjacent to the pit at S_{16} . Regarding S_{15} , the optimal action in the deterministic case is to move up; however, in the stochastic case, moving up involves a 10% chance of falling into the pit. The optimal strategy is therefore to (repeatedly) attempt to move left, with a 10% chance of slipping up but no chance of falling into the pit. Similarly, in S_{17} , the optimal action in the deterministic case is to move down, but this move has a 10% chance of slipping to the left into the pit; the optimal action is to (repeatedly) attempt to move right until the agent moves down to S_{23} .

To see this policy in action, we can simulate a realization of a path starting from the initial state. To illustrate the stochastic nature of the environment, consider the path starting from S_6 . While the intended move was “down”, a slippage to the right occurs.

Simulated Path (Stochastic Policy)

